

Q1 Load json via stage and flatten array of sessions

1a Execution of create table,create stage,create file format , copy into statements

```
6 CREATE OR REPLACE TABLE student_courses_raw (
7   student VARIANT
8 );
9
10 CREATE OR REPLACE STAGE student_courses_stage;
11 Ctrl+I to generate
12 CREATE OR REPLACE FILE FORMAT ff_json_courses
13   TYPE = JSON
14   STRIP_OUTER_ARRAY = TRUE;
15
16   list @student_courses_stage;
17
18
19 COPY INTO student_courses_raw
20 FROM @student_courses_stage
21 FILE_FORMAT = (FORMAT_NAME = ff_json_courses)
22 on_error=continue;
```

Results (2 minutes ago)

Table	Chart	1 row	38ms	⬇	🕒
name	size	md5	last_modified		
student_courses_stage/cust2.json.gz	624	266e478f60892ff1fc760fd8fc506b98	Fri, 9 Jan 2026 05:03:50 GMT		

Json data loaded to stage

```
1 Row(s) produced. Time Elapsed: 1.346s
darshuhegde2002#COMPUTE_WH@SNOWPARK_DB.PUBLIC>put file:///C:/snowpark/darshan/cust2.json @student_courses_stage;
+-----+-----+-----+-----+-----+-----+-----+-----+
| source | target | source_size | target_size | source_compression | target_compression | status | message |
+-----+-----+-----+-----+-----+-----+-----+-----+
| cust2.json | cust2.json.gz | 6031 | 624 | NONE | GZIP | UPLOADED | |
+-----+-----+-----+-----+-----+-----+-----+-----+
1 Row(s) produced. Time Elapsed: 1.110s
darshuhegde2002#COMPUTE_WH@SNOWPARK_DB.PUBLIC>
```

```
18
19 COPY INTO student_courses_raw
20 FROM @student_courses_stage
21 FILE_FORMAT = (FORMAT_NAME = ff_json_courses)
22 on_error=continue;
23
```

results (just now)

Table

Chart

🔍

📄

1 row

ⓘ

1.9s

⬇

🕒

id	file	status	:	# rows_parsed	# rows_loaded	# error_limit	# errors_seen	first_error
1	student_courses_stage	LOADED		5	5	5	0	null

Select query showing raw variant data

```
23
24 select * from student_courses_raw;
```

results (just now)

Table	Chart	5 rows	641ms	⬇	🕒
{?} STUDENT					
1	{ "meta": { "plan": "Pro", "region": "IN", "signup_ts": "2024-12-15T08:30:00Z" }, "name": "Anita Rao", "sessions": [{ "active": false, "session_id": "SES-00-1				
2	{ "meta": { "plan": "Basic", "region": "IN", "signup_ts": "2024-11-20T10:05:00Z" }, "name": "Rahul Menon", "sessions": [{ "active": false, "session_id": "SES				
3	{ "meta": { "plan": "Pro", "region": "IN", "signup_ts": "2024-10-02T12:00:00Z" }, "name": "Meera Iyer", "sessions": [{ "active": false, "session_id": "SES-20				
4	{ "meta": { "plan": "Enterprise", "region": "IN", "signup_ts": "2024-12-01T09:45:00Z" }, "name": "Vikram Singh", "sessions": [{ "active": false, "session_id":				

1b select query using lateral flatten on the session array with nested field extraction.

```
26 SELECT
27     session.value:session_id::STRING AS session_id,
28     session.value:active::BOOLEAN AS active,
29     session.value:start_ts::TIMESTAMP_NTZ AS start_ts
30 FROM student_courses_raw,
31 LATERAL FLATTEN(input => student:sessions) AS session;
32
```

1c final output query displaying flattened columns

```
26 SELECT
27     session.value:session_id::STRING AS session_id,
28     session.value:active::BOOLEAN AS active,
29     session.value:start_ts::TIMESTAMP_NTZ AS start_ts
30 FROM student_courses_raw,
31 LATERAL FLATTEN(input => student:sessions) AS session;
32
```

results (just now)

Table Chart 40 rows 854ms

SESSION_ID	ACTIVE	START_TS
SES-00-001	2.5%	01/01/2025
SES-00-002	2.5%	17/02/2025
+38 more		

1	SES-00-001	FALSE	2025-01-01 09:00:00.000
2	SES-00-002	TRUE	2025-01-02 10:00:00.000
3	SES-00-003	TRUE	2025-01-03 11:00:00.000
4	SES-00-004	FALSE	2025-01-04 12:00:00.000
5	SES-00-005	TRUE	2025-01-05 13:00:00.000

1d (user metadata+sessionID,start_ts,active)

```
26 SELECT
27     student:name::STRING AS name,
28     student:user_id::STRING AS user_id,
29     student:meta:region::STRING AS region,
30     session.value:session_id::STRING AS session_id,
31     session.value:active::BOOLEAN AS active,
32     session.value:start_ts::TIMESTAMP_NTZ AS start_ts
33 FROM student_courses_raw,
34 LATERAL FLATTEN(input => student:sessions) AS session;
35
```

Results (just now)

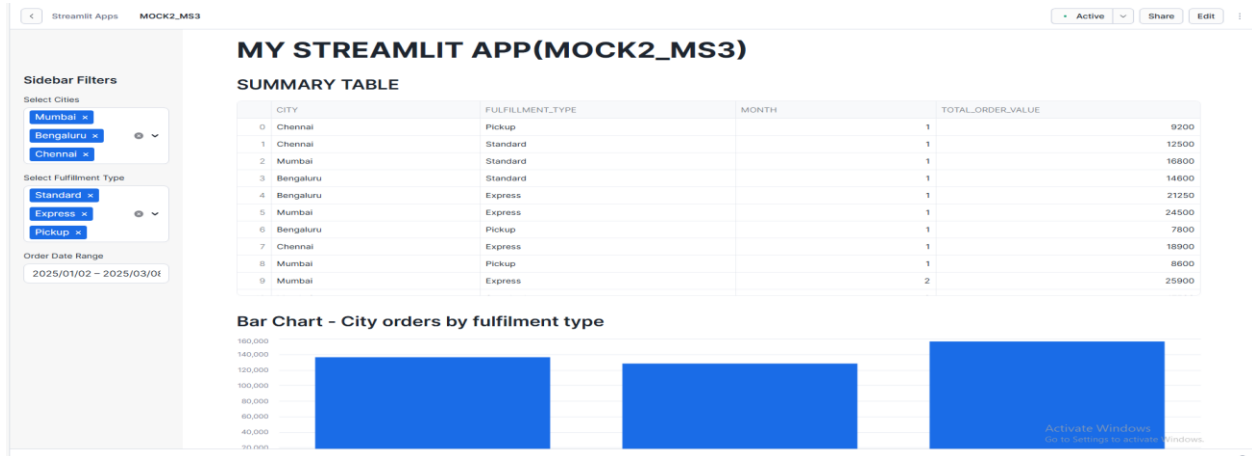
Table Chart 40 rows 95ms

NAME	USER_ID	REGION	SESSION_ID	ACTIVE	START_TS
Anita Rao	U1001		SES-00-001	2.5%	01/01/2025
Meera Iyer	U1002		SES-00-002	2.5%	17/02/2025
+3 more	+3 more	100.0%	+38 more		

19	Meera Iyer	U1003	IN	SES-20-003	TRUE	2025-01-23 11:00:00.000
20	Meera Iyer	U1003	IN	SES-20-004	FALSE	2025-01-24 12:00:00.000
21	Meera Iyer	U1003	IN	SES-20-005	TRUE	2025-01-25 13:00:00.000
22	Meera Iyer	U1003	IN	SES-20-006	TRUE	2025-01-26 14:00:00.000

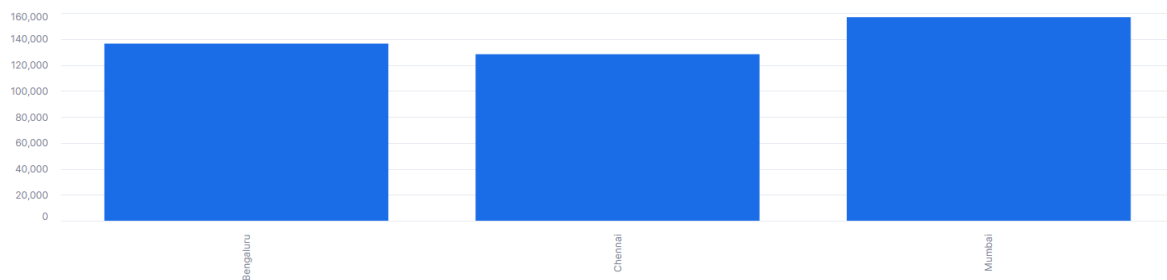
Q2 Build streamlit App for city-wise orders by fulfilment type (Interactive filters and native visuals)

2a App running with the sidebar filters visible (city , fulfillment type, date range)

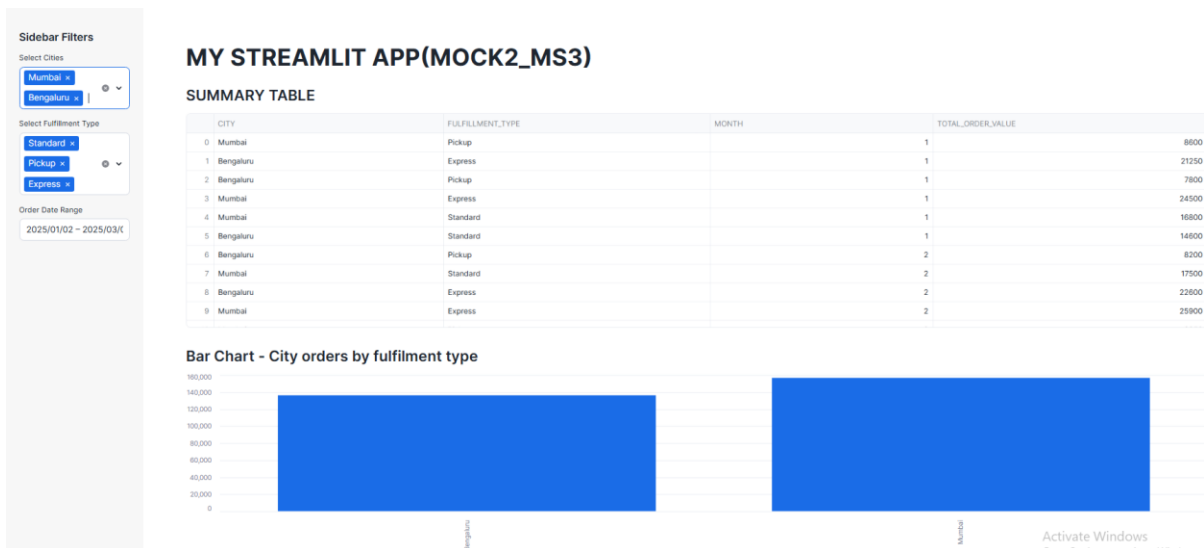


2b bar chart showing city orders by fulfillment type(filtered)

Bar Chart - City orders by fulfillment type



After applying filter



2c pie chart showing order value shared by city (filtered)

Select Cities

Mumbai ×

Bengaluru ×

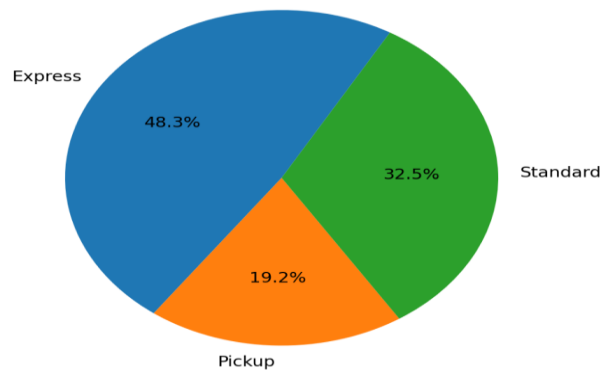
Chennai ×

Select Fulfillment Type

Standard ×

Express ×

Pickup ×



After the filter application

Select Cities

Chennai ×

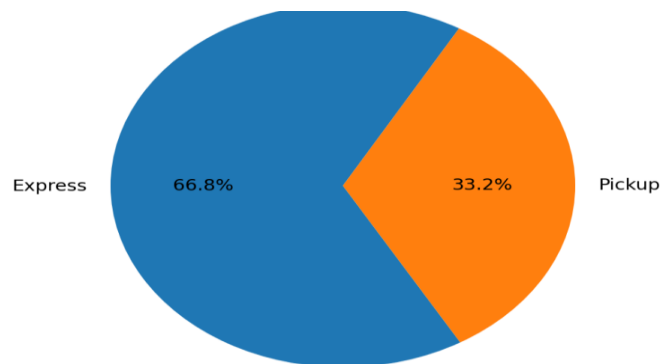
Select Fulfillment Type

Express ×

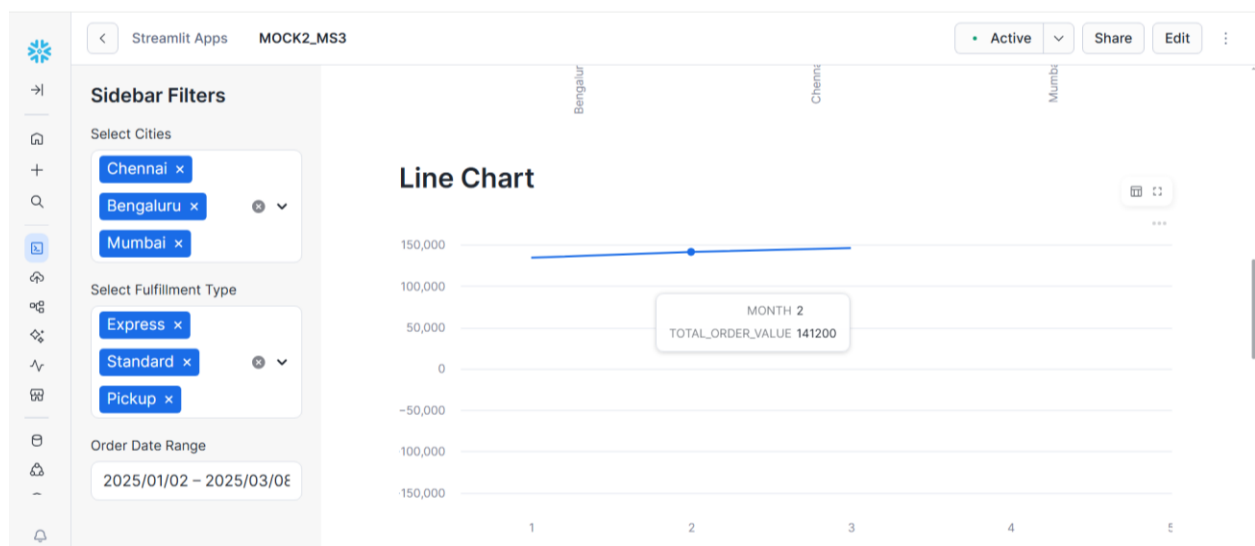
Pickup ×

Order Date Range

2025/01/02 – 2025/03/08



2d line chart + summary tables showing time series trends and top fulfillment types



After applying filter

→|

+

Q

Sidebar Filters

Select Cities

Chennai ×

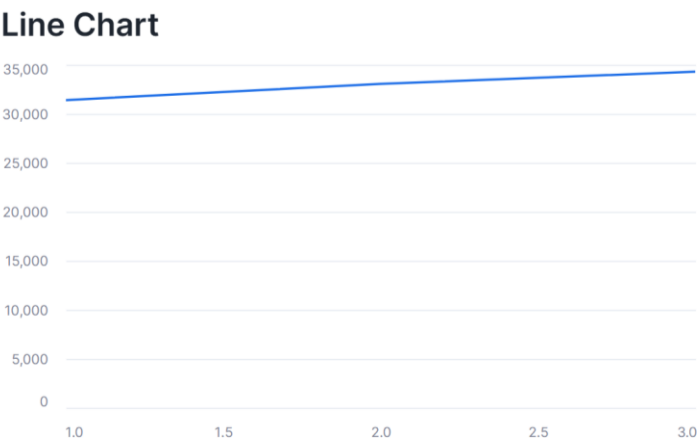
Select Fulfillment Type

Express ×

Standard ×

Order Date Range

2025/01/02 – 2025/03/08



Q3 Configure the cortex search service and cortex search preview for semantic document retrieval

3a create table and insert statements execution

```
39
40 CREATE OR REPLACE TABLE help_library (
41   doc_id STRING,
42   title STRING,
43   body STRING,
44   category STRING,
45   source_uri STRING
46 );
47
48 -- 2) Insert sample documents (procedures, guides, policies, FAQs)
49 INSERT INTO help_library (doc_id, title, body, category, source_uri) VALUES
50 ('HL-001', 'Onboarding Procedure',
51  'Follow the onboarding steps: account setup, MFA enablement, device registration, and policy acknowledgment.',
52  'Procedure', 'kb/procedures/onboarding.txt'),
53 ('HL-002', 'VPN Setup Guide',
```

Results (1 minute ago)

Table

Chart

1 row ⓘ 1.1s

#	number of rows inserted
1	5

3b cortex search preview configuration creation screenshot (index and config setup)

```
70 create or replace cortex search service mock2_ms3
71 on body
72 warehouse=compute_wh
73 target_lag='1 minute'
74 as
75 select doc_id,body from help_library;
```

Results (just now)

Table

Chart

1 row ⓘ 3.2s

A	status
1	Cortex search service MOCK2_MS3 successfully created.

```
/o
77 show cortex search services;
```

Results (just now)

Table

Chart

created_on DESC ✕

5 rows ⓘ 109ms

#	created_on	name	database_name	schema_name	target_lag	warehouse	search_column	att
1	2026-01-08 21:27:05.662 -0800	MOCK2_MS3	SNOWPARK_DB	PUBLIC	1 minute	COMPUTE_WH	BODY	

3c semantic query execution screenshot with two different queries, showing top matches the scores

82

83

84

85

86

87

88

89

```
select snowflake.cortex.search_preview(  
  'mock2_ms3',  
  '{  
    "query": "How to setup vpn?",  
    "columns": ["doc_id", "body"],  
    "limit": 2  
  }' )as matched;
```

Results (just now)

TableChart

1 row198ms

	MATCHED
1	<pre>{ "results": [{ "@scores": { "text_match": 0.2254451, "cosine_similarity": 0.6815475 }, "body": "To set up the VPN: install the client, import the"</pre>

MATCHED

```
{  
  "results": [  
    {  
      "@scores": {  
        "text_match": 0.2254451,  
        "cosine_similarity": 0.6815475  
      },  
      "body": "To set up the VPN: install the client, import the"
```

82

83

84

85

86

87

88

89

```
select snowflake.cortex.search_preview(  
  'mock2_ms3',  
  '{  
    "query": "What is the process for onboarding?",  
    "columns": ["doc_id", "body"],  
    "limit": 2  
  }' )as matched;
```

Results (just now)

TableChart

1 row192ms

	MATCHED
1	<pre>{ "results": [{ "@scores": { "text_match": 0.11687302, "cosine_similarity": 0.5338741 }, "body": "Follow the onboarding steps: account setup, MFA enablement,"</pre>

MATCHED

```
{  
  "results": [  
    {  
      "@scores": {  
        "text_match": 0.11687302,  
        "cosine_similarity": 0.5338741  
      },  
      "body": "Follow the onboarding steps: account setup, MFA enablement,"
```

3d Final tabular output screenshot with columns query ,matched , snippet ,score ,category,docid

93
94
95

```
select body as query,snowflake.cortex.search_preview(  
'mock2_ms3',  
|'{'
```

Results (just now)

TableChart

1 row1.0s

	QUERY	MATCHED	CATEGORY	DOC_ID
1	To set up the VPN: install the cli	{"results":[{"@scores":{"text_mat	Guide	HL-002

Q4 Join two snowpark dataframes, aggregate , apply having like filter and sort descending

4a snowpark session initialization and dataframe creation from sales_lineitem and product_dim

```
1  from snowflake.snowpark import Session
2  from snowflake.snowpark.functions import *
3
4  session=Session.builder\
5  |      |      .config("connection_name","default").create()
6
7  product_df=session.table("product_dim")
8  product_df.show()
9
10 sales_df=session.table("sales_lineitems")
11 sales_df.show()
```

```
• (darshan) PS C:\snowpark> & c:/snowpark/darshan/Scripts/python.exe c:/snowpark/darshan/mock2_ms3.py
```

"PRODUCT_ID"	"PRODUCT_NAME"	"CATEGORY"
Q001	Smartphone X	Electronics
Q002	Noise-Cancel Headset	Electronics
Q003	Ergo Keyboard	Electronics
Q004	Mesh Office Chair	Furniture
Q005	Standing Work Desk	Furniture
Q006	Air Fryer	Home & Kitchen
Q007	Espresso Machine	Home & Kitchen
Q008	LED Desk Lamp	Home & Kitchen
Q009	Bookshelf 5-Tier	Furniture
Q010	Portable Blender	Home & Kitchen

"ORDER_ID"	"ORDER_DATE"	"PRODUCT_ID"	"QTY"	"UNIT_PRICE"
L1001	2025-01-05	Q001	3	35000.00
L1002	2025-01-06	Q002	5	8000.00
L1003	2025-01-06	Q003	4	4500.00
L1004	2025-01-07	Q004	2	12000.00
L1005	2025-01-08	Q005	1	29000.00
L1006	2025-01-08	Q006	7	6000.00
L1007	2025-01-09	Q007	2	26000.00
L1008	2025-01-09	Q008	10	3500.00
L1009	2025-01-10	Q009	3	15000.00
L1010	2025-01-10	Q010	6	3200.00

4b join step showing snowpark dataframe .join and revenue generation

```
joined_df=product_df.join(sales_df,product_df["product_id"]==sales_df["product_id"],"inner")
joined_df.show()
```

(darshan) PS C:\snowpark> & c:/snowpark/darshan/Scripts/python.exe c:/snowpark/darshan/mock2_ms3.py

"l_0000_PRODUCT_ID"	"PRODUCT_NAME"	"CATEGORY"	"ORDER_ID"	"ORDER_DATE"	"r_0001_PRODUCT_ID"	"QTY"	"UNIT_PRICE"
Q001	Smartphone X	Electronics	L1001	2025-01-05	Q001	3	35000.00
Q002	Noise-Cancel Headset	Electronics	L1002	2025-01-06	Q002	5	8000.00
Q003	Ergo Keyboard	Electronics	L1003	2025-01-06	Q003	4	4500.00
Q004	Mesh Office Chair	Furniture	L1004	2025-01-07	Q004	2	12000.00
Q005	Standing Work Desk	Furniture	L1005	2025-01-08	Q005	1	29000.00
Q006	Air Fryer	Home & Kitchen	L1006	2025-01-08	Q006	7	6000.00
Q007	Espresso Machine	Home & Kitchen	L1007	2025-01-09	Q007	2	26000.00
Q008	LED Desk Lamp	Home & Kitchen	L1008	2025-01-09	Q008	10	3500.00
Q009	Bookshelf 5-Tier	Furniture	L1009	2025-01-10	Q009	3	15000.00
Q010	Portable Blender	Home & Kitchen	L1010	2025-01-10	Q010	6	3200.00

4c aggregation and having like filters step group by ,agg,filter

```
sale = joined_df.with_column(
    "SALES_VALUE",
    col("QTY") * col("UNIT_PRICE")
)

#sale.show()

aggr = sale.group_by(
    "CATEGORY",
    "PRODUCT_NAME"
).agg(
    sf_sum("SALES_VALUE").alias("TOTAL_REVENUE"),
    count("*").alias("TOTAL_COUNT")
)

aggr.show()
```

"CATEGORY"	"PRODUCT_NAME"	"TOTAL_SALES"
Electronics	Smartphone X	208000.00
Electronics	Ergo Keyboard	96400.00
Furniture	Mesh Office Chair	71900.00
Furniture	Standing Work Desk	118000.00
Home & Kitchen	Air Fryer	204300.00
Home & Kitchen	Espresso Machine	180100.00
Electronics	Noise-Cancel Headset	126200.00
Home & Kitchen	LED Desk Lamp	85100.00
Furniture	Bookshelf 5-Tier	135400.00
Home & Kitchen	Portable Blender	44600.00

Application of filter

```
fil=aggr.filter(col("TOTAL_REVENUE")>100000).alias("FILTERED")
fil.show()
```

"CATEGORY"	"PRODUCT_NAME"	"TOTAL_REVENUE"
Electronics	Smartphone X	208000.00
Furniture	Standing Work Desk	118000.00
Home & Kitchen	Air Fryer	204300.00
Home & Kitchen	Espresso Machine	180100.00
Electronics	Noise-Cancel Headset	126200.00
Furniture	Bookshelf 5-Tier	135400.00

4d Final sorted output table with category product name total revenue order count

```
final=fil.sort(col("TOTAL_REVENUE").desc())
final.show()
```

"CATEGORY"	"PRODUCT_NAME"	"TOTAL_REVENUE"	"TOTAL_COUNT"
Electronics	Smartphone X	208000.00	3
Home & Kitchen	Air Fryer	204300.00	3
Home & Kitchen	Espresso Machine	180100.00	3
Furniture	Bookshelf 5-Tier	135400.00	3
Electronics	Noise-Cancel Headset	126200.00	3
Furniture	Standing Work Desk	118000.00	3